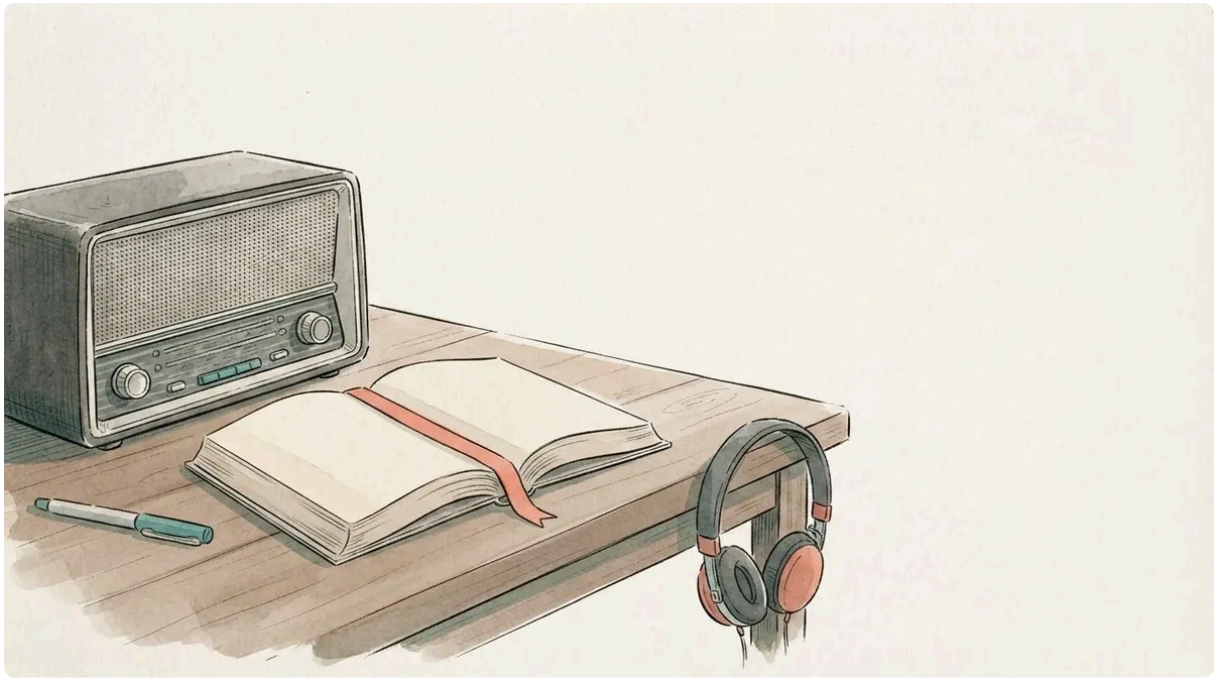
Audio on a static site: build-time TTS and the script it reads from

Emil Lindfors | July 28, 2026



The audio project came from the realization that my partner can get through a research article much quicker when she listens to the text while reading instead of just reading, so I wanted to explore it for myself with LLM audio generation. So I cloned my voice and tried it out.

The result is at the top of this page: about twelve and a half minutes of me, synthesised at build time and committed to git next to the PDF. The rest of the back catalogue is switched off. I ran the pipeline over all eight posts once to see what it came to, got 97 minutes and 45 MB, and have not decided whether I want that in the repo. This is how it works, what went wrong on the way, and why it is shelved for now.

What the research says

My partner is dyslexic, and the evidence for text-to-speech and dyslexia is better than I expected. Dyslexia is common: the estimates in Peterson & Pennington's review (2012) start at 5 percent and go up with how broadly it is defined. Wood et al. (2017) pooled the studies on read-aloud tools and reading disabilities and found a real improvement in

comprehension. The mechanism is plain: decoding costs effort, and the effort comes out of the same budget as understanding. Take the decoding away and the budget goes to the meaning.

The finding that changed my design is older. Montali & Lewandowski (1996) tested less skilled readers three ways: text alone, audio alone, and text with the spoken word highlighted as it went. The bimodal version won on comprehension and on how the readers felt about the task. That is exactly what my partner does with a research article, and it is stage four for me. It is not built yet. It is the reason the pipeline emits an intermediate script instead of going straight from markdown to MP3, because word-level highlighting needs a text to align against.

One more thing. Synthetic speech costs the listener more effort than a human voice does, and the worse the voice, the more it costs (Duffy & Pisoni, 1992). That review is from 1992, when synthetic meant a formant synthesiser, but the direction holds. `speechSynthesis` in the browser is free and I am not using it.

Why build time

Three options: synthesise in the browser, synthesise on request in a Cloudflare Function, or synthesise at build time and commit the result.

The browser option is free and immediate, and it sounds like a 2009 satnav. The Function option puts a GPU in the request path for a page that is otherwise static files on a CDN. So: build time, committed, the same way the post PDFs already work. Cloudflare Pages runs its own `zola build` on push, and anything not in the repo does not survive the trip.

Storage is fine. Mono at 64 kbps gives 7.2 MB for the longest post, which runs 15:46, and 45 MB for the whole archive. Cloudflare's limits are 25 MiB per file and 20,000 files. Git will carry that. If it stops being fine, R2 is the documented escape hatch. If you are on any static host, do the same: generate once, commit, and keep the GPU out of the request path.

The script comes first

Roughly a third of this blog by volume is code fences, tables, ASCII diagrams and a rendered reference list. None of that is listenable. Hand raw markdown to a TTS model and you get a voice reading `#[derive(Debug)]` out loud, and then reading a DOI.

So there is a step before synthesis. `site-tools speech` derives a spoken script and writes it to `static/speech/<slug>.txt`, committed and diffable. When the audio sounds wrong, that file is where I look, and it is much cheaper to read than to listen to.

In the post	In the script
Fenced code block	`Rust code block, 14 lines. See the article.`
Box-drawing diagram	`Diagram, 15 lines. See the article.`
Table	`Table, 7 rows. See the article.`
`![alt](src)`	`Figure: alt.` (dropped if alt is empty)
`$$…$$`	`Equation.`
`## References` and everything after	dropped
`[text](url)`	`text`
`et al.`	`and others`

Structure survives as whitespace. One blank line between blocks is a paragraph gap, two is a section break, and the synthesis step turns those into 0.45 and 0.9 seconds of silence. Because the structure is in the readable file, there is no separate manifest describing it.

The transformation has to be fence-aware. Two of these posts quote Tera *inside* code fences, and a stripper that did not track fences would delete the examples those posts exist to show.

What the script got wrong

I generated all eight scripts and read them. Reading them is how I found the following. Listening would have taken 97 minutes.

`client_max_body_size` came out as `clientmaxbodysize`. I was stripping `_` as an emphasis marker without checking whether it was inside a word. Same for `codex_turn_token_usage` and `genai.usage.total_cost`. The fix is to treat inline code differently from prose: inside backticks `_` becomes a space, outside it is emphasis and gets dropped.

`~/.fraktal/config.toml` came out as `/.fraktal/config.toml`, because I was stripping `~` as a strikethrough marker. Only `~~` is strikethrough. A lone tilde is a home directory.

Bullet lists were merging into one block, so three list items became one 40-second sentence with no breath in it. Each item is its own block now.

And in the two posts about Zola templates, inline Tera like `{{<citation key="smith2024" num="1" />}}` was going through to the model verbatim. There is no good pronunciation for that. It now becomes “this tag”, which reads acceptably in a sentence (“in a post, this tag renders as a clickable 1”) and fires six times in the worst post. Posts that are mostly syntax can opt out entirely with `extra.skip_audio`, mirroring the `extra.skip_citations` opt-out I already had. Most of the back catalogue has it set.

None of these were crashes. They were plausible output that happened to be wrong. If you build one of these, read the scripts before you pay for the audio.

Picking a model and a voice

I am running Fish Audio S2.1 Pro. Two reasons: it is good, and I can move it onto my own GPU later without changing anything but a URL. The pipeline knows a base URL, a model string and a voice, and all three come from `.env`. The voice is a Fish `reference_id`, and mine points at a clone of my own voice.

Cost turned out not to be a factor at all. The whole archive is 84,327 characters of speech script. At the going rates that is:

Service	Rate	Whole archive
Google Cloud Standard	$4/M	$0.34
OpenAI `tts-1`	$15/M	$1.26
Cloudflare Aura-1	$15/M	$1.26
ElevenLabs Flash	$50/M	$4.22

I paid nothing, because `s2.1-pro-free` is free until the end of August 2026. But even the expensive column is one coffee for the entire back catalogue, so the decision came down to voice quality and whether I could self-host it later.

There are two backend shapes in the code, chosen by an enum: Fish's `POST /v1/tts`, and the OpenAI `POST /v1/audio/speech` shape, which also covers Kokoro-FastAPI and the OpenAI-compatible Fish wrappers. A closed set of two, so static dispatch and no trait object. HTTP goes through `curl`, the way the newsletter sender already does. `site-tools` depends on `toml` and one git crate, and a build-time task does not justify an async runtime.

Blocks, not posts

Each block goes to the API on its own instead of the whole post in one call. Three reasons, and all three turned out to matter:

- S2.1 Pro is autoregressive, and long generations drift.
- A block that comes back wrong is retried alone.
- Editing one paragraph costs one API call, not seventeen minutes of GPU time.

If you take one thing from this post for your own pipeline, take the block size. The blocks come back as WAV and get joined with ffmpeg's concat demuxer, then a single encode to MP3. Stitching MP3s instead would put a frame-boundary gap at every block and add a second generation of encoding loss.

The silence between blocks is cut from a real block:

```
ffmpeg -i block.wav -af volume=0 -t 0.45 gap-paragraph.wav
```

That guarantees the sample rate and channel count match the blocks exactly, which the concat demuxer requires. I could have probed the model output and constructed matching silence, but this is one command and cannot drift.

Then one pass over the whole thing:

```
ffmpeg -f concat -safe 0 -i concat.txt \
  -af loudnorm=I=-16:TP=-1.5:LRA=11 \
  -ac 1 -codec:a libmp3lame -b:a 64k out.mp3
```

`loudnorm` at -16 LUFS is the podcast convention. Without it, volume drifts between posts synthesised weeks apart. Measured after the fact, the first post came out at -16.6 LUFS integrated. Close enough.

Two bugs in this part were mine and both would have shipped silently. An HTTP 200 is not proof of audio: an API can answer 200 with a JSON error body, and I was caching that as a valid block and concatenating it. It checks for a RIFF header now. And when a block failed all its retries, curl had already written the error body to the output path, where the next run would find it and treat it as cached audio.

The gate

Every block is hashed, and the whole script is hashed. If the script hash matches what is recorded in the sidecar and the MP3 exists, the command makes no network calls at all.

This is what keeps the GPU out of the critical path. `build.sh` runs the audio step behind a check for ffmpeg and a reachable endpoint, and treats failure as a warning, the same posture the citation step already takes. If my GPU box is asleep and I want to fix a typo and deploy, nothing blocks. The committed MP3 ships unchanged.

FNV-1a for the hashing. It is a cache key, not a signature. The question it answers is “is this block byte-identical to the one I already paid to synthesise”.

The player

The player sits above the article body, not in the sidebar. Someone who needs it should not have to hunt for it, and the sidebar is hidden on mobile anyway. It renders only when the sidecar JSON exists, which Zola picks up with `load_data`. That is also how it stays off everywhere else: with no sidecar there is no player and no empty space where one would be.

Native `<audio controls>`, so it is keyboard-reachable and works with the media keys and with JavaScript off. The only scripted part is the speed buttons, and the choice is remembered. If you listen at 1.5×, you want that on the next post too.

One detail I would have got wrong without testing it in a browser: setting `playbackRate` is not enough. `load()` resets it to `defaultPlaybackRate`, and with `preload="none"` the audio is fetched long after the restore runs. It survives in Chrome today. It did not look like it would survive a reload, so both get set.

There is a line under the player saying the narration is synthetic and that code blocks and tables are summarised. That seemed like the minimum.

Shelved, for now

Word-level highlighting is the part with the research behind it and it is not built. Fish returns audio bytes and no timings, so the plan is a WhisperX forced-alignment pass over the generated MP3 using the speech script as the known transcript. That is alignment, not recognition, which is a much easier problem, and it needs the script file that already exists.

A podcast feed is the other obvious thing, and it needs the back catalogue committed first. That would be about two hours of audio, with an Atom feed already sitting next to it.

`speech-lexicon.toml` exists for pronunciation overrides and has one entry in it: nginx, which every TTS model I have tried says as "en-jinx". "Typst" appears 39 times across the archive and I do not know yet whether it needs a line in there, because I still have not listened to all of what I generated.

But I wasn't very satisfied with it, so it's been shelved for now until I find a better TTS service or find that it's actually useful for a blog.

It sounds synthetic, and after the 1992 review above I do not want to hand a dyslexic reader a voice that makes the reading harder. The pipeline stays. The gate means it costs nothing to keep. When a better model turns up, the change is one line in `.env`, and the scripts are already committed and already read.

References

- Peterson, R. L., & Pennington, B. F. "Developmental dyslexia". *The Lancet*, vol. 379, no. 9830, pp. 1997-2007, 2012. doi:10.1016/s0140-6736(12)60198-6
- Wood, S. G., Moxley, J. H., Tighe, E. L., & Wagner, R. K. "Does Use of Text-to-Speech and Related Read-Aloud Tools Improve Reading Comprehension for Students With Reading Disabilities? A Meta-Analysis". *Journal of Learning Disabilities*, vol. 51, no. 1, pp. 73-84, 2017. doi:10.1177/0022219416688170
- Montali, J., & Lewandowski, L. "Bimodal Reading: Benefits of a Talking Computer for Average and Less Skilled Readers". *Journal of Learning Disabilities*, vol. 29, no. 3, pp. 271-279, 1996. doi:10.1177/002221949602900305
- Duffy, S. A., & Pisoni, D. B. "Comprehension of Synthetic Speech Produced by Rule: A Review and Theoretical Interpretation". *Language and Speech*, vol. 35, no. 4, pp. 351-389, 1992. doi:10.1177/002383099203500401